



UNIVERSIDADE
ESTADUAL DO CEARÁ



Intelligent
Networks and
Systems Group

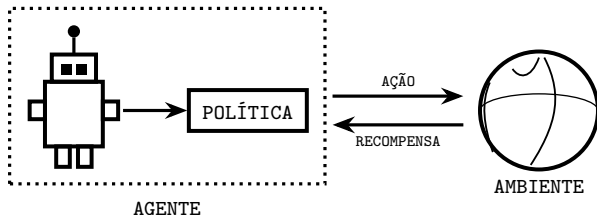
DOUTORADO EM CIÊNCIA DA COMPUTAÇÃO — REDES DE
COMPUTADORES

Modelo de Decisão Baseado em Lógica *Fuzzy* e
Aprendizado por Reforço Profundo
para o *Offloading* de Tarefas
em Redes Veiculares 5G

Antônio Sérgio de Sousa Vieira

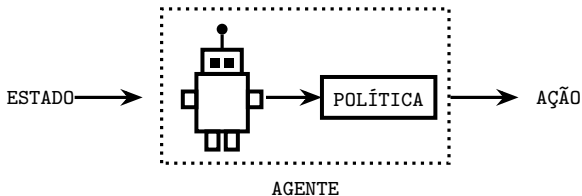
APRENDIZADO POR REFORÇO

UMA ABORDAGEM DE APRENDIZADO DE MÁQUINA NA QUAL UM AGENTE APRENDE A TOMAR DECISÕES REALIZANDO AÇÕES EM UM AMBIENTE E RECEBENDO RECOMPENSAS OU PENALIDADES, MELHORANDO GRADUALMENTE SEU COMPORTAMENTO PARA MAXIMIZAR A RECOMPENSA ACUMULADA.



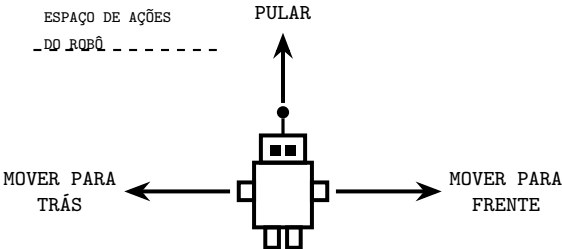
AGENTE

UMA ENTIDADE AUTÔNOMA QUE PERCEBE SEU AMBIENTE POR MEIO DE SENSORES, TOMA DECISÕES COM BASE EM SEU ESTADO E POLÍTICA ATUAIS, E REALIZA AÇÕES PARA MAXIMIZAR A RECOMPENSA ACUMULADA AO LONGO DO TEMPO.



AÇÃO

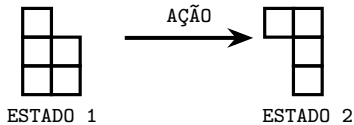
UMA ESCOLHA OU MOVIMENTO ESPECÍFICO DISPONÍVEL PARA O AGENTE NO ESTADO ATUAL, QUE CAUSA UMA TRANSIÇÃO PARA UM NOVO ESTADO NO AMBIENTE.



AMBIENTE

O MUNDO EXTERNO COM O QUAL O AGENTE INTERAGE. RECEBE AS AÇÕES DO AGENTE, REALIZA TRANSIÇÕES PARA NOVOS ESTADOS E FORNECE SINAIS DE RECOMPENSA DE VOLTA AO AGENTE.

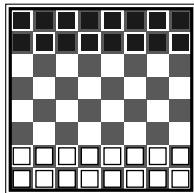
-- AMBIENTE DO TETRIS --



ESTADO

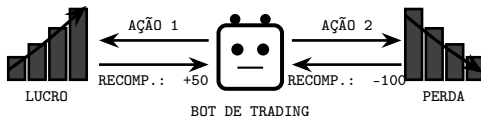
UMA DESCRIÇÃO COMPLETA DA SITUAÇÃO OU CONFIGURAÇÃO ATUAL DO AMBIENTE EM UM MOMENTO ESPECÍFICO NO TEMPO.

ESTADO INICIAL DO AMBIENTE DE XADREZ



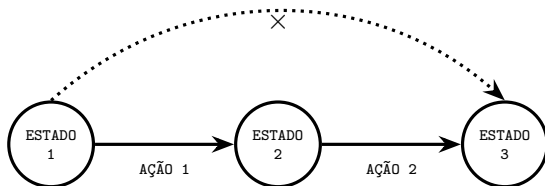
RECOMPENSA

UM SINAL DE FEEDBACK NUMÉRICO RECEBIDO IMEDIATAMENTE APÓS TOMAR UMA AÇÃO,
INDICANDO O QUÃO BOA OU RUIM FOI AQUELA AÇÃO NO ESTADO ATUAL.



PROPRIEDADE DE MARKOV

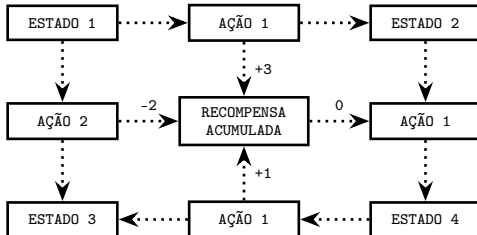
UMA PROPRIEDADE QUE AFIRMA QUE O ESTADO FUTURO DE UM PROCESSO DEPENDE APENAS DO ESTADO ATUAL, E NÃO DA SEQUÊNCIA DE EVENTOS QUE LEVOU A ELE.



MDP

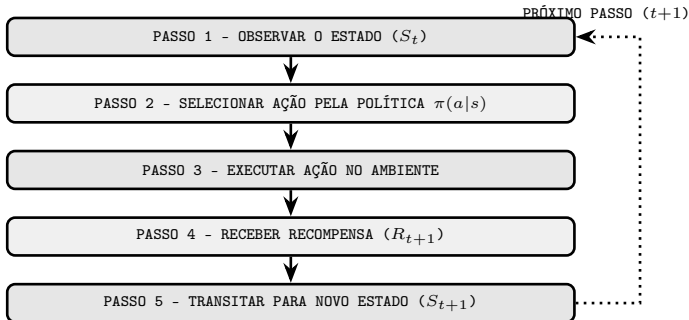
PROCESSO DE DECISÃO DE MARKOV

UM FRAMEWORK MATEMÁTICO PARA TOMADA DE DECISÃO SEQUENCIAL SOB INCERTEZA, DEFINIDO POR ESTADOS, AÇÕES, PROBABILIDADES DE TRANSIÇÃO E RECOMPENSAS, COM O OBJETIVO DE ENCONTRAR UMA POLÍTICA QUE MAXIMIZE A RECOMPENSA ACUMULADA ESPERADA.



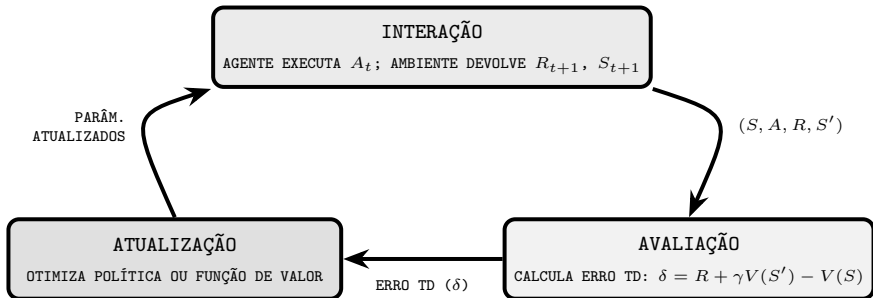
CICLO DO MDP

A INTERAÇÃO ENTRE AGENTE E AMBIENTE REPETE-SE ITERATIVAMENTE EM 5 PASSOS SEQUENCIAIS.



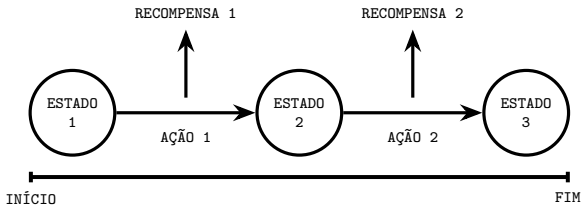
CICLO DE TREINAMENTO EM RL

DURANTE O TREINAMENTO, O CICLO DE INTERAÇÃO GANHA UMA ETAPA CRUCIAL: A ATUALIZAÇÃO. APÓS AGIR E OBSERVAR (S, A, R, S') , O AGENTE CALCULA O ERRO DE SUA PREVISÃO E ATUALIZA ITERATIVAMENTE SUA POLÍTICA OU FUNÇÃO DE VALOR, MELHORANDO SEU COMPORTAMENTO A CADA PASSO.



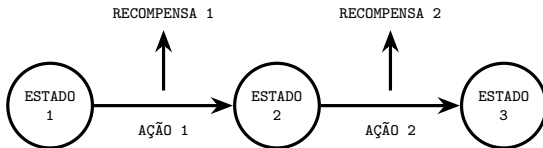
EPISÓDIO

UMA TRAJETÓRIA FINITA DE ESTADOS, AÇÕES E RECOMPENSAS QUE COMEÇA EM UM ESTADO INICIAL E TERMINA EM UM ESTADO TERMINAL.



RETORNO

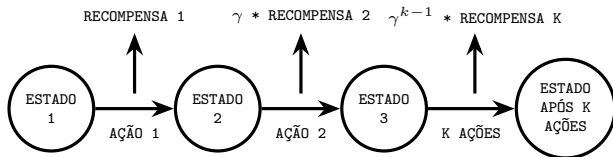
O RETORNO G_t É A SOMA DE TODAS AS RECOMPENSAS FUTURAS QUE O AGENTE RECEBE A PARTIR DO INSTANTE t , ATÉ O FIM DO EPISÓDIO.



$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots = \sum_{k=0}^{\infty} R_{t+k+1}$$

FATOR DE DESCONTO

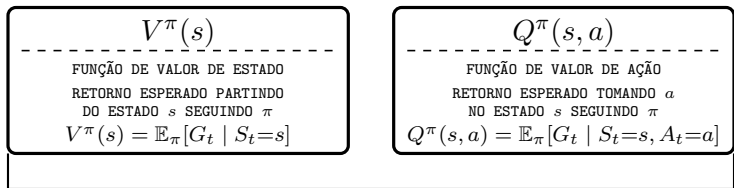
UM VALOR (γ) ENTRE 0 E 1 QUE DETERMINA O QUANTO AS RECOMPENSAS FUTURAS VALEM EM RELAÇÃO ÀS IMEDIATAS. $\gamma = 0$ SIGNIFICA QUE APENAS A RECOMPENSA IMEDIATA IMPORTA; $\gamma = 1$ SIGNIFICA QUE TODAS AS RECOMPENSAS FUTURAS TÊM O MESMO PESO.



$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

FUNÇÕES DE VALOR

ESTIMAM O RETORNO ESPERADO A LONGO PRAZO - QUÃO BOM É ESTAR EM UM ESTADO OU TOMAR UMA AÇÃO SEGUINDO UMA POLÍTICA π .



VANTAGEM: $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$

EQUAÇÃO DE BELLMAN

UMA RELAÇÃO RECURSIVA QUE EXPRESSA O VALOR DE UM ESTADO COMO A RECOMPENSA IMEDIATA MAIS O VALOR DESCONTADO DO PRÓXIMO ESTADO.

$$V^\pi(s) = R(s) + \gamma \cdot V^\pi(s')$$

- $V^\pi(s)$ = VALOR DO ESTADO ATUAL
- $R(s)$ = RECOMPENSA IMEDIATA
- γ = FATOR DE DESCONTO
- $V^\pi(s')$ = VALOR DO PRÓXIMO ESTADO

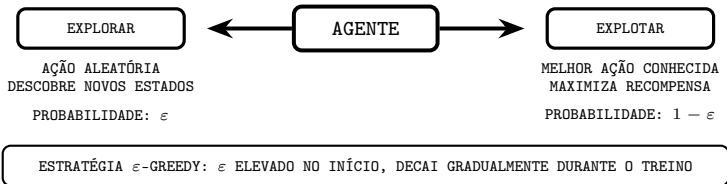
EQUAÇÃO DE OTIMALIDADE (Q^*):

$$Q^*(s, a) = R(s, a) + \gamma \cdot \boxed{\max_{a'}} Q^*(s', a')$$

O OPERADOR $\boxed{\max_{a'}}$ ESCOLHE A AÇÃO FUTURA DE MAIOR VALOR, GARANTINDO A POLÍTICA ÓTIMA.

DILEMA EXPLORAÇÃO-EXPLOTAÇÃO

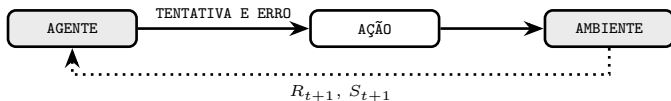
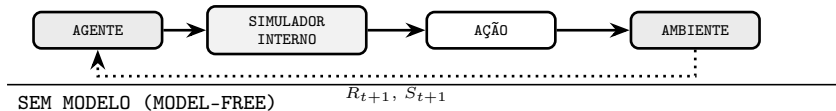
O AGENTE DEVE EQUILIBRAR EXPLORAR O AMBIENTE (DESCOBRIR NOVOS ESTADOS) E EXPLOTAR O CONHECIMENTO ATUAL (MAXIMIZAR RECOMPENSA IMEDIATA).



ABORDAGENS: COM MODELO VS. SEM MODELO

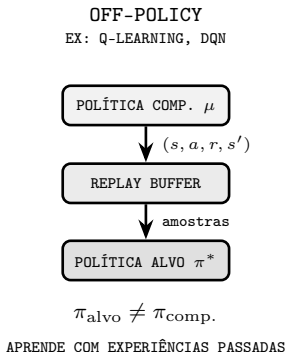
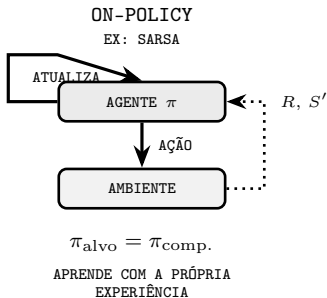
EM ABORDAGENS COM MODELO (MODEL-BASED), O AGENTE USA OU APRENDE UM MODELO DO AMBIENTE PARA SIMULAR FUTUROS E PLANEJAR AÇÕES. MÉTODOS SEM MODELO (MODEL-FREE) APRENDEM DIRETAMENTE POR TENTATIVA E ERRO, SEM CONHECER AS REGRAS INTERNAS DO AMBIENTE.

COM MODELO (MODEL-BASED)



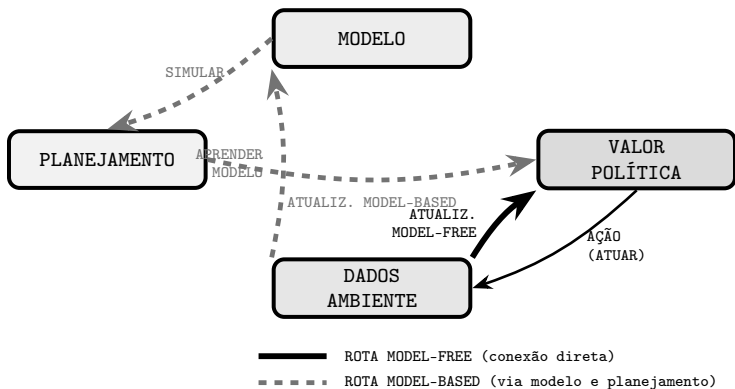
APRENDIZADO: ON-POLICY VS. OFF-POLICY

NO APRENDIZADO ON-POLICY (EX: SARSA), O AGENTE APRENDE E AGE COM A MESMA POLÍTICA. NO OFF-POLICY (EX: Q-LEARNING, DQN), O AGENTE OTIMIZA UMA POLÍTICA ALVO ENQUANTO EXPLORA COM UMA POLÍTICA DIFERENTE, PODENDO REUTILIZAR EXPERIÊNCIAS PASSADAS DE UM REPLAY BUFFER.

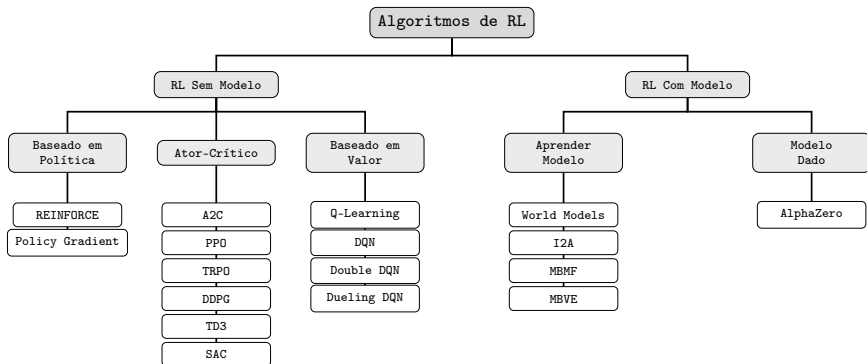


ARQUITETURA GERAL E MODULAR DO RL

O RL PODE SER VISTO COMO QUATRO BLOCOS MODULARES: DADOS/AMBIENTE, MODELO, PLANEJAMENTO E VALOR/POLÍTICA. O ALGORITMO ESCOLHIDO APENAS LIGA OU DESLIGA CONEXÕES ENTRE ELES: MÉTODOS MODEL-FREE CONECTAM DADOS DIRETAMENTE À POLÍTICA; MÉTODOS MODEL-BASED PASSAM PELO MODELO E PELO PLANEJAMENTO PARA ATUALIZAR A POLÍTICA COM MAIS EFICIÊNCIA.



ALGORITMOS DE RL



Q-LEARNING

UM ALGORITMO CLÁSSICO SEM MODELO QUE APRENDE A FUNÇÃO DE VALOR ÓTIMA $Q^*(s, a)$ DIRETAMENTE DA EXPERIÊNCIA, ATUALIZANDO UMA TABELA USANDO A EQUAÇÃO DE BELLMAN DE OTIMALIDADE.

Q-TABELA

	a_1	a_2	a_3
s_1	0.2	0.5	0.1
$s_2 \leftarrow$	0.8	0.3	0.6
s_3	0.1	0.7	0.4

$\max_{a'}$

REGRA DE ATUALIZAÇÃO

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

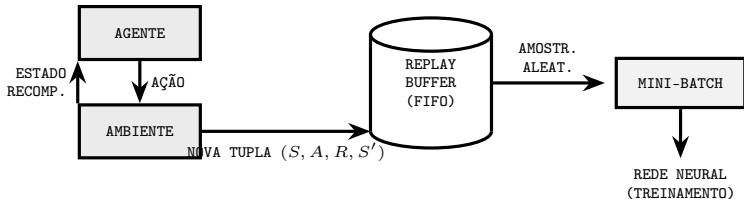
α : taxa de aprendizado γ : fator de desconto
 $\max_{a'}$: melhor ação futura no estado s'

REPLAY BUFFER

REPETIÇÃO DE EXPERIÊNCIA

PROBLEMA: TREINAR DIRETO NA SEQUÊNCIA $s_t, s_{t+1}, s_{t+2}, \dots$ FAZ A REDE VICIAR NO PADRÃO RECENTE E ESQUECER EXPERIÊNCIAS ANTERIORES, POIS AMOSTRAS CONSECUTIVAS SÃO ALTAMENTE CORRELACIONADAS.

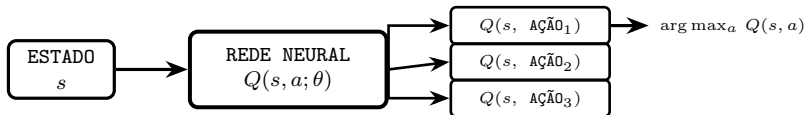
SOLUÇÃO: GUARDAR AS ÚLTIMAS N TUPLAS (S, A, R, S') NUM BUFFER E SORTEAR UM MINI-BATCH ALEATÓRIO A CADA ATUALIZAÇÃO. AMOSTRAS DISTANTES NO TEMPO SÃO MISTURADAS $\Rightarrow$ GRADIENTES ESTÁVEIS E CADA EXPERIÊNCIA É REUTILIZADA VÁRIAS VEZES.



DQN

DEEP Q-NETWORK

ESTENDE O Q-LEARNING SUBSTITUINDO A Q-TABELA POR UMA REDE NEURAL $Q(s, a; \theta)$, PERMITINDO LIDAR COM ESPAÇOS DE ESTADOS CONTÍNUOS E DE ALTA DIMENSÃO (EX.: PIXELS DE JOGOS ATARI).



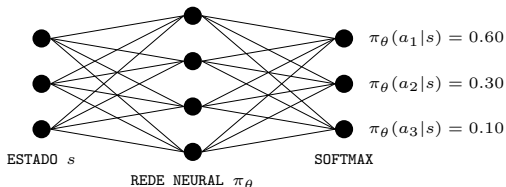
INOVAÇÕES DO DQN SOBRE O Q-LEARNING CLÁSSICO

- **REPLAY BUFFER:** armazena (s, a, r, s') e amostra aleatoriamente para quebrar correlações temporais.
- **REDE ALVO (θ^-):** segunda cópia da rede, congelada por K passos. O alvo $y = R + \gamma \max_{a'} Q(s', a'; \theta^-)$ usa θ^- em vez de θ , evitando que o alvo mude a cada passo e cause instabilidade no treinamento.

GRADIENTE DE POLÍTICA

REINFORCE

OTIMIZA DIRETAMENTE A POLÍTICA $\pi_\theta(a|s)$ AJUSTANDO θ (PESOS E VIESES DA REDE NEURAL) NA DIREÇÃO DO GRADIENTE DO RETORNO ESPERADO. NÃO PRECISA ESTIMAR $Q(s, a)$ (APROXIMADOR DE VALOR = REDE QUE APRENDE O QUANTO VALE CADA AÇÃO).



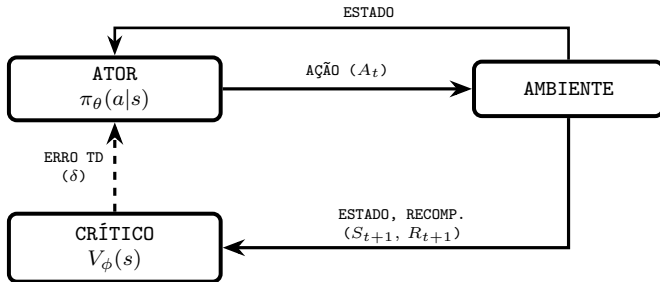
ATUALIZAÇÃO (REINFORCE)

$$\theta \leftarrow \theta + \alpha \cdot G_t \cdot \nabla_\theta \log \pi_\theta(a_t|s_t)$$

- Por que log? Não dá para derivar π_θ diretamente pela amostra. Usa-se $\nabla_\theta \pi = \pi \cdot \nabla_\theta \log \pi$ para reescrever o gradiente como uma esperança - estimável por Monte Carlo.
- G_t : soma de recompensas futuras descontadas. Se $G_t > 0$, θ é ajustado para aumentar $\pi_\theta(a_t|s_t)$; se $G_t < 0$, para diminuir.

ATOR-CRÍTICO

O ATOR APRENDE A POLÍTICA $\pi_{\theta}(a|s)$ ENQUANTO O CRÍTICO ESTIMA A FUNÇÃO DE VALOR $V_{\phi}(s)$. O CRÍTICO USA O ERRO TD $\delta = R_{t+1} + \gamma V_{\phi}(S_{t+1}) - V_{\phi}(S_t)$ PARA AVALIAR SE A AÇÃO FOI MELHOR OU PIOR DO QUE O ESPERADO, ENVIANDO ESSE SINAL PARA GUIAR A ATUALIZAÇÃO DO ATOR E REDUZIR A VARIÂNCIA DO APRENDIZADO.



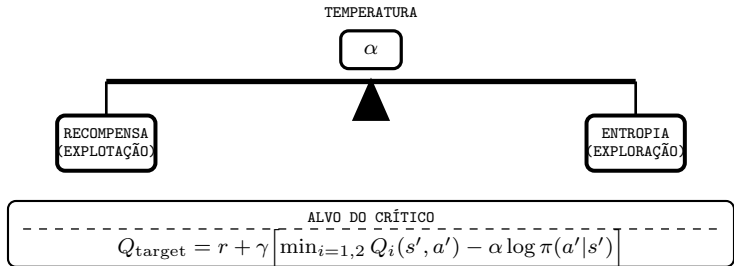
- CRÍTICO: ESTIMA O RETORNO ESPERADO $V_{\phi}(S_t)$ PARA CALCULAR O ERRO TD (δ).

SE $\delta > 0$: AÇÃO LEVOU A UM ESTADO MELHOR QUE O ESPERADO $\rightarrow$ ATOR REFORÇA A AÇÃO.

SE $\delta < 0$: AÇÃO LEVOU A UM ESTADO PIOR QUE O ESPERADO $\rightarrow$ ATOR INIBE A AÇÃO.

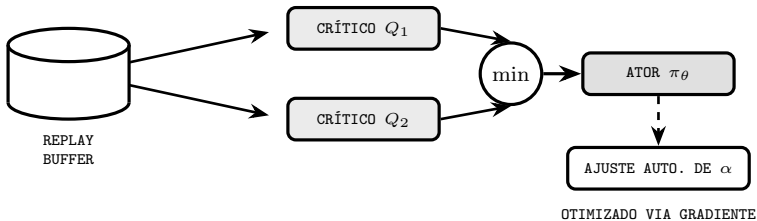
SOFT ACTOR-CRITIC (SAC)

O SAC É UM ALGORITMO ATOR-CRÍTICO AVANÇADO *OFF-POLICY* QUE TREINA UMA POLÍTICA ESTOCÁSTICA. SUA PRINCIPAL INOVAÇÃO É ADICIONAR A ENTROPIA DA POLÍTICA À FUNÇÃO DE VALOR: AO MAXIMIZAR CONJUNTAMENTE RECOMPENSA E ENTROPIA, O SAC GARANTE ALTA EXPLORAÇÃO, ROBUSTEZ E EFICIÊNCIA DE DADOS.



SAC: ARQUITETURA E EFICIÊNCIA

O SAC USA DUAS REDES CRÍTICAS SEPARADAS (DOUBLE Q-LEARNING) PARA EVITAR SUPERESTIMAÇÃO, TOMANDO SEMPRE O VALOR MÍNIMO. O COEFICIENTE α É OTIMIZADO AUTOMATICAMENTE VIA GRADIENTE DURANTE O TREINO, ELIMINANDO A NECESSIDADE DE AJUSTE MANUAL DESTE HIPERPARÂMETRO.





FOFF: an energy-efficient task offloading in VEC-enabled vehicular networks using fuzzy TOPSIS

Antônio Sérgio de Sousa Vieira^{1,2} · Alisson Barbosa de Souza^{1,3} · Joaquim Celestino Júnior¹

Received: 19 February 2025 / Revised: 8 May 2025 / Accepted: 11 May 2025 / Published online: 19 September 2025
© The Author(s), under exclusive license to Springer Science+Business Media, LLC part of Springer Nature 2025

Abstract

In modern vehicular network environments, the demand for efficient data processing is crucial. However, this demand can exceed the computational capacity of the vehicles' On-Board Unit Interconnected (OBUI). One way to overcome these limitations is through edge devices with high computational capacity. Vehicular Edge Computing (VEC) facilitates the execution of these processing demands; however, latency remains a critical challenge. With advancements in 5G technology, research efforts have increasingly focused on determining whether tasks should be processed locally or offloaded to edge servers. Beyond latency considerations, energy consumption presents a significant concern for both OBUI and edge devices. This study introduces Fuzzy Offloading (FOFF), an innovative approach designed to balance these competing factors. Experiments show that FOFF reduces energy consumption and latency, achieving 72.11% energy savings and 0.61% lower processing time compared to the best greedy offloading approach considered. These results highlight FOFF's efficiency in vehicular environments.

Keywords Task offloading · Energy optimization · Fuzzy topsis · Vehicular network

1 Introduction

With the popularization and cost reduction of various communication technologies and high-capacity computational devices, interest in the development of vehicles capable of autonomous driving has also grown. However, creating technology for this purpose proves to be quite complex and challenging [1], specially within context of Intelligent Transportation Systems (ITS), where integrating

autonomous vehicles with existing infrastructure and other vehicles demands innovative solutions to ensure efficiency, usefulness, reliability and safety [2].

Building on this context, the development and evolution of 5G NR (Fifth-Generation Wireless Technology New Radio) and 6G (Sixth-Generation Wireless Technology) bring new application possibilities in the vehicular scenario. These solutions provide transmission speeds exceeding 20 Gbps and 1 Tbps [3, 4], respectively. Additionally, transmission latency, which in 5G NR is approximately 1 ms [3], will be reduced to the microsecond range in 6G (approximately 5 μ s) [4], driving the adoption of autonomous vehicles (AVs) in our daily lives. Thus, these technologies foster the development of real-time applications essential for the safe and efficient operation of autonomous vehicles in complex environments. However, as latency decreases, a new challenge emerges: the need to process the massive influx of data quickly and efficiently.

As high communication latency becomes less of an issue for real-time applications, it is important to consider solutions capable of processing large volumes of data with minimal response times while simultaneously reducing energy costs [5]. In this context, several proposals have been presented, particularly those emphasizing the use of more

✉ Antônio Sérgio de Sousa Vieira
sergio.vieira@ufec.edu.br

Alisson Barbosa de Souza
alison@ufec.br

Joaquim Celestino Júnior
joaquim.celestino@ufec.br

¹ Intelligent Networks and Systems Group (INETSYS), State University of Ceará, Av. Dr. Silas Mangaba, 1700, Fortaleza, CE 60714-903, Brazil

² Federal Institute of Education, Science and Technology, Av. da Universidade, 102, Itapipoca, CE 62500-000, Brazil

³ Federal University of Ceará, Av. José F. Queiroz, 5003, Quixadá, CE 63902-580, Brazil



Deep learning approaches for computation offloading in edge computing: A critical review

Sapthagiri Miriyala¹ · Venkata Ramireddy Chirra¹

Received: 21 June 2025 / Accepted: 11 February 2026 / Published online: 24 February 2026
 © The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

Abstract

The paradigm shift from centralized cloud computing to Multi-Access Edge Computing (MEC) has been necessitated by the exponential growth of the Internet of Things (IoT) and the stringent latency requirements of 5G/6G applications. However, the inherent dynamism of edge environments, characterized by fluctuating channel conditions, heterogeneous device capabilities, and finite energy budgets, renders traditional static optimization techniques insufficient. This survey provides a critical review of Deep Learning (DL) and Deep Reinforcement Learning (DRL) as pivotal enablers for autonomous computation offloading. This work systematically dissects the algorithmic trade-offs between Value-Based (e.g., DQN) and Policy-Gradient (e.g., PPO, SAC, A3C) architectures, evaluating their suitability for continuous control, multi-agent coordination, and resource-constrained inference. We further bridge the gap between theoretical algorithms and practical telecommunications deployment by mapping DRL strategies to European Telecommunications Standards Institute (ETSI) MEC specifications and Open Radio Access Network (O-RAN) architectural standards. The review analyzes the “optimization trade-offs” inherent in offloading, specifically the conflict between service immediacy (latency) and resource conservation (energy/security). Finally, we synthesize key challenges hindering large-scale adoption, including the need for lightweight “extreme edge” models, robustness against non-stationary environments, and privacy-preserving federated learning, thereby providing a rigorous roadmap for the evolution of self-optimizing, intelligent edge ecosystems.

Keywords Multi-Access Edge Computing · Deep Reinforcement Learning · Computation Offloading · Resource Allocation · Federated Learning · IoT

Nomenclature

A2C	Advantage Actor-Critic
A3C	Asynchronous Advantage Actor-Critic
AI	Artificial Intelligence
BS	Base Station
CC	Cloud Computing
CNN	Convolutional Neural Network
CSI	Channel State Information
CTDE	Centralized Training Decentralized Execution
DAG	Directed Acyclic Graph
DDPG	Deep Deterministic Policy Gradient
DDQN	Double Deep Q-Network
DL	Deep Learning
DNN	Deep Neural Network
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning

ETSI	European Telecommunications Standards Institute
FDRL	Federated Deep Reinforcement Learning
FL	Federated Learning
GNN	Graph Neural Network
GPU	Graphics Processing Unit
IID	Independent and Identically Distributed
IIoT	Industrial Internet of Things
IoT	Internet of Things
IoUAV	Internet of Unmanned Aerial Vehicles
IoV	Internet of Vehicles
LLM	Large Language Model
LSTM	Long Short-Term Memory
MADDPG	Multi-Agent Deep Deterministic Policy Gradient
MADRL	Multi-Agent Deep Reinforcement Learning
MAML	Model-Agnostic Meta-Learning
MAPPO	Multi-Agent Proximal Policy Optimization
MARL	Multi-Agent Reinforcement Learning
MCC	Mobile Cloud Computing

Venkata Ramireddy Chirra
 venkataramireddy.chirra@vitap.ac.in

¹ VIT-AP University, Amaravati, India

ORIGINALIDADE E POSICIONAMENTO NO ESTADO DA ARTE

- Contribuição Central: Arquitetura híbrida SAC+TOPSIS para descarregamento de tarefas em redes VEC.
- TOPSIS: Comprime o espaço de estados variável (N candidatos) em um vetor compacto de dimensão fixa (6D) -- invariante à escala.
- SAC: Política estocástica com entropia máxima para lidar com a não estacionariedade do ambiente veicular.
- Publicado: *FOFF: an energy-efficient task offloading in VEC-enabled vehicular networks using fuzzy TOPSIS* [Vieira et al., 2025].

LACUNAS NO ESTADO DA ARTE E RESPOSTA DA PROPOSTA

LACUNA 1 [Miriayala e Chirra, 2026]:

- Literatura usa *flat vectors* de dimensão fixa.
- Ignora densidade variável de veículos e mudanças de topologia.

⇒ RESPOSTA: TOPSIS como compressor de estado -- saída 6D fixo independente de N .

LACUNA 2 [Miriayala e Chirra, 2026]:

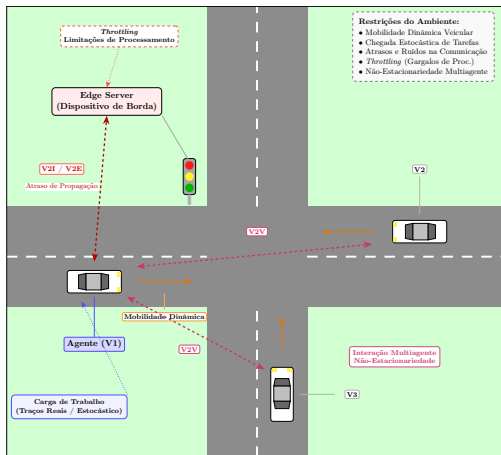
- Políticas treinadas de forma estática divergem em ambientes de alta mobilidade.

⇒ RESPOSTA: SAC+TOPSIS com regularização por entropia previne colapso da política e mantém exploração contínua guiada por múltiplos critérios.

AMBIENTE DE SIMULAÇÃO

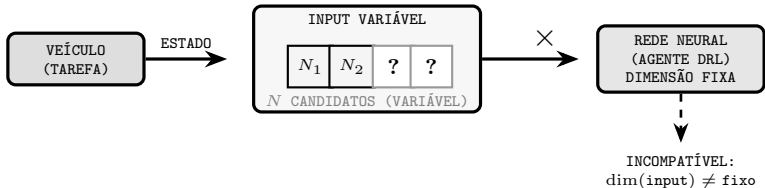
CRUZAMENTO VEICULAR ESTOCÁSTICO COM COMUNICAÇÃO V2I

(VEÍCULO-INFRAESTRUTURA) E V2V (VEÍCULO-VEÍCULO). INCORPORA MOBILIDADE DINÂMICA, CHEGADA ESTOCÁSTICA DE TAREFAS, ATRASOS DE PROPAGAÇÃO E LIMITAÇÕES DE PROCESSAMENTO (*THROTTLING*).



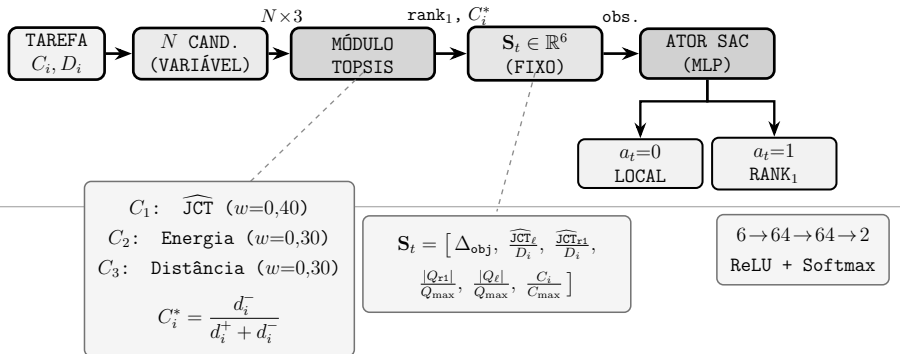
O PROBLEMA: DIMENSIONALIDADE VARIÁVEL EM VEC

O DESCARREGAMENTO EM REDES VEC LIDA COM ALTA MOBILIDADE: O NÚMERO DE NÓS CANDIDATOS (RSUs E VEÍCULOS V2V) MUDA A TODO INSTANTE. COMO REDES NEURAIS DE DRL EXIGEM ENTRADAS DE DIMENSÃO FIXA, AS SOLUÇÕES TRADICIONAIS FALHAM AO GENERALIZAR PARA DIFERENTES DENSIDADES SEM RETREINAMENTOS. O DESAFIO: CRIAR UMA REPRESENTAÇÃO DE ESTADO INVARIANTE À ESCALA.



A PROPOSTA: PIPELINE SAC-TOPSIS

DIVISÃO DE RESPONSABILIDADES: O TOPSIS RESPONDE “QUAL CANDIDATO?” (ELIMINA A VARIABILIDADE DIMENSIONAL); O SAC RESPONDE “DESCARREGAR?” (DECISÃO BINÁRIA, APRENDIDA COM ENTROPIA MÁXIMA E REPLAY BUFFER).



TOPSIS: CRITÉRIOS, ESCORE E ESTADO 6D

CRITÉRIOS DO TOPSIS

Crit.	Descrição	Peso
C_1	$\widehat{JCT}$ (latência estimada)	0,40
C_2	Energia total (J)	0,30
C_3	Distância física (m)	0,30

ESCORE DE PROXIMIDADE

$$C_i^* = \frac{d_i^-}{d_i^+ + d_i^-}$$

rank_1 = candidato de maior C_i^*

VETOR DE ESTADO $S_t \in \mathbb{R}^6$

$$S_t = \left[\Delta_{\text{obj}}, \frac{\widehat{JCT}_\ell}{D_i}, \frac{\widehat{JCT}_{r1}}{D_i}, \frac{|Q_{r1}|}{Q_{\max}}, \frac{|Q_\ell|}{Q_{\max}}, \frac{C_i}{C_{\max}} \right]$$

$$\Delta_{\text{obj}} = (\widehat{JCT}_\ell - \widehat{JCT}_{r1}) / D_i$$

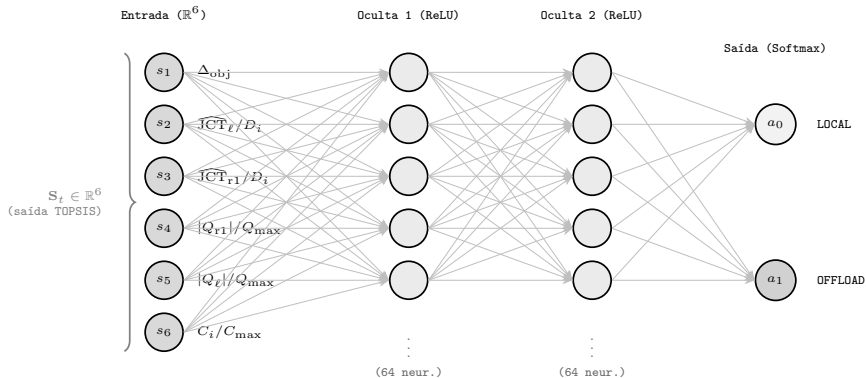
> 0: offload mais rápido < 0: local mais rápido

$s_1 = \Delta_{\text{obj}}$	redução relativa de custo (offload vs. local)
$s_2 = \widehat{JCT}_\ell / D_i$	urgência local
$s_3 = \widehat{JCT}_{r1} / D_i$	urgência do offload
$s_4 = Q_{r1} / Q_{\max}$	congestion. do candidato
$s_5 = Q_\ell / Q_{\max}$	congestion. local
$s_6 = C_i / C_{\max}$	complexidade da tarefa

$$\text{modo}_t = \begin{cases} \text{LOCAL} & a_t = 0 \\ \text{rank}_1 \in \{\text{RSU}, \text{V2V}\} & a_t = 1 \end{cases}$$

ATOR SAC: REDE NEURAL MLP

ARQUITETURA $6 \rightarrow 64 \rightarrow 64 \rightarrow 2$ RELU + SOFTMAX



Param.	Descrição	Valor
η	Taxa de aprendizado (Adam)	3×10^{-4}
γ	Fator de desconto	0,99
α_{ent}	Coefficiente de entropia	0,10
$ B $	Tamanho do replay buffer	50 000
batch	Tamanho do mini-batch	1 024

ALGORITMO 1 - TOPSIS (RANQUEAMENTO MULTICRITÉRIO)

```
Entrada: Matriz de decisão  $\mathbf{X} \in \mathbb{R}^{m \times n}$ ; vetor de pesos  $\mathbf{w}$   
Saída : Vetor de proximidade  $\mathbf{C}^*$  (alternativas ranqueadas)  
// Passos 1-2: Normalização  $L_2$  por coluna e ponderação  
1 for  $j = 1$  to  $n$  do  
2   for  $i = 1$  to  $m$  do  
3      $r_{ij} \leftarrow x_{ij} / \sqrt{\sum_{k=1}^m x_{kj}^2}$   
4      $v_{ij} \leftarrow w_j \cdot r_{ij}$   
5   end  
   // Passo 3: Soluções ideal positiva  $A^+$  e negativa  $A^-$   
6   if critério  $j$  é de custo then  
7      $A_j^+ \leftarrow \min_i v_{ij}; \quad A_j^- \leftarrow \max_i v_{ij}$   
8   else  
9      $A_j^+ \leftarrow \max_i v_{ij}; \quad A_j^- \leftarrow \min_i v_{ij}$   
10  end  
11 end  
   // Passos 4-5: Distâncias  $L_2$  (Euclidianas) e coeficiente de  
   proximidade  
12 for  $i = 1$  to  $m$  do  
13    $d_i^+ \leftarrow \sqrt{\sum_{j=1}^n (v_{ij} - A_j^+)^2}; \quad d_i^- \leftarrow \sqrt{\sum_{j=1}^n (v_{ij} - A_j^-)^2}$   
14    $C_i^* \leftarrow d_i^- / (d_i^+ + d_i^-)$   
15 end  
16 return Ordenação decrescente de  $\mathbf{C}^*$ 
```

ALGORITMO 2 - SAC-TOPSIS (INFERÊNCIA)

Entrada: Tarefa i : $C_i, D_i, \mathbf{x}_i$;
 N candidatos $\mathcal{M}$;
MLP π_θ ;
 $Q_\ell, Q_{r1}, Q_{\max}, C_{\max}$

Saída : Ação $a_t \in \{0, 1\}$: local ou offload para rank₁

// Passo 1 - Matriz de decisão ($N \times 3$)

- 1 **for** $k = 1$ **to** N **do**
- 2 | $x_{k1} \leftarrow \widehat{\text{JCT}}(m_k)$; $x_{k2} \leftarrow E(m_k)$; $x_{k3} \leftarrow \|\mathbf{x}_i - \mathbf{x}_{m_k}\|$
- 3 **end**

// Passo 2 - Ranqueamento via Algoritmo 1 (todos critérios de custo)

- 4 $\mathbf{w} \leftarrow [0,40; 0,30; 0,30]$; $\mathbf{C}^* \leftarrow \text{Algoritmo1}(\mathbf{X}, \mathbf{w})$; rank₁ $\leftarrow \arg \max_k C_k^*$

// Passo 3 - Estado compacto 6D

- 5 $\Delta_{\text{obj}} \leftarrow (\widehat{\text{JCT}}_\ell - \widehat{\text{JCT}}_{r1}) / D_i$
- 6 $\mathbf{S}_t \leftarrow [\Delta_{\text{obj}}, \widehat{\text{JCT}}_\ell / D_i, \widehat{\text{JCT}}_{r1} / D_i, Q_{r1} / Q_{\max}, Q_\ell / Q_{\max}, C_i / C_{\max}]$

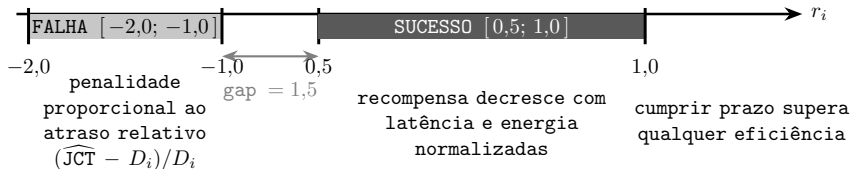
// Passo 4 - Inferência e decisão

- 7 $a_t \leftarrow \arg \max_a \pi_\theta(a | \mathbf{S}_t)$
- 8 **if** $a_t = 0$ **then**
- 9 | **return** LOCAL
- 10 **else**
- 11 | **return** OFFLOAD para rank₁
- 12 **end**

RECOMPENSA *DEADLINE*-AWARE

$$r_i = \begin{cases} 1,0 - \frac{1}{4} \left(\frac{\widehat{\text{JCT}}_i}{D_i} + \frac{E_i}{E_i^{\max}} \right), & \widehat{\text{JCT}}_i \leq D_i \quad (\in [0,5, 1,0]) \\ -1,0 - \min \left(1,0, \frac{\widehat{\text{JCT}}_i - D_i}{D_i} \right), & \widehat{\text{JCT}}_i > D_i \quad (\in [-2,0, -1,0]) \end{cases}$$

$E_i^{\max} = \max(E_i^{\ell}, E_i^{\text{rsu}} \cdot \mathbf{1}_{\text{viável}}, E_i^{\text{v}2\text{v}} \cdot \mathbf{1}_{\text{viável}})$ - energia máxima entre os modos viáveis (por tarefa, por instante)



ESTUDO DE ABLAÇÃO: DESIGN EXPERIMENTAL

CADA VARIANTE REMOVE OU SUBSTITUI UM ÚNICO COMPONENTE DA PROPOSTA COMPLETA. PERMITE QUANTIFICAR A CONTRIBUIÇÃO CAUSAL ISOLADA DE CADA DECISÃO DE DESIGN: ESPAÇO DE ESTADO, ESPAÇO DE AÇÃO, FUNÇÃO DE RECOMPENSA E PAPEL DO TOPSIS.

POLÍTICA	ESTADO	AÇÃO	RECOMPENSA	TOPSIS
SAC-BASE	11D (bruto)	3 classes	WSM	--
SAC-A1	11D (bruto)	binária	WSM	--
SAC-S1	6D (manual)	3 classes	WSM	--
SAC-R1	11D (bruto)	3 classes	Deadline-aware	--
SAC-T1	6D (TOPSIS)	3 classes	Deadline-aware	Feature
SAC-TOPSIS	6D (TOPSIS)	binária	Deadline-aware	Decisão

3 classes: $a_t \in \{\text{LOCAL}, \text{RSU}, \text{v2v}\}$ Binária: $a_t \in \{\text{LOCAL}, \text{OFFLOAD} \rightarrow \text{rank}_1\}$

WSM: $r \in [-1, 1]$ proporcional ao custo (JCT + energia + fila). Problema: não penaliza violação de prazo - tarefa atrasada com custo baixo recebe recompensa positiva.

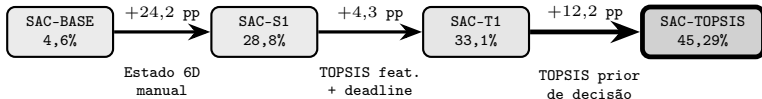
DEADLINE-AWARE: dois regimes separados por lacuna de 1,5 unidades:

- $\text{JCT} \leq D_i$ (sucesso): $r \in [0,5; 1,0]$ - quanto menor o custo, maior a recompensa.
- $\text{JCT} > D_i$ (falha): $r \in [-2,0; -1,0]$ - penalidade cresce com o atraso.

Efeito: cumprir o prazo é *sempre* melhor que qualquer falha, independente do custo.

ESTUDO DE ABLAÇÃO: DECOMPOSIÇÃO CAUSAL

POLÍTICA	TAXA DEADLINE (%)	JCT (s)	ENERGIA (J)	Δ pp
SAC-BASE	4,6	57,1	2422	--
SAC-A1	39,2	13,7	2106	+34,6
SAC-S1	28,8	39,3	2028	+24,2
SAC-R1	38,4	13,4	3153	+33,8
SAC-T1	33,1	11,5	2612	+28,5
SAC-TOPSIS	45,29	6,6	2582	+40,7



$$\Delta_{\text{estado}} > \Delta_{\text{TOPSIS-decisão}} > \Delta_{\text{reward+feature}} \gg \Delta_{\text{ação-binária}}$$

ESTUDO DE ABLAÇÃO: ANÁLISE DOS ACHADOS

ACHADO 1 - AÇÃO BINÁRIA ESTÁTICA JÁ SUPERA O BASELINE

SAC-A1 (11D, binária, regra $VEC \succ V2V$): 39,2% $\rightarrow$ +34,6 pp sobre SAC-Base (4,6%).

SIMPLES REDUÇÃO DO ESPAÇO DE BUSCA TEM EFEITO EXPRESSIVO.

PORÉM NÃO ALCANÇA SAC-TOPSIS (45,29%): A REGRA ESTATICA IGNORA CONDIÇÕES DINÂMICAS DE FILA E DISTÂNCIA NO MOMENTO DA DECISÃO.

ACHADO 2 - TOPSIS COMO DECISÃO VALE $3\times$ MAIS QUE COMO FEATURE

SAC-T1 (TOPSIS como feature): +4,3 pp sobre SAC-S1.

SAC-TOPSIS (TOPSIS como prior de destino): +12,2 pp sobre SAC-T1.

EFICÁCIA DO TOPSIS É QUASE $3\times$ MAIOR QUANDO ELE ASSUME O CONTROLE DO DESTINO DO QUE QUANDO APENAS INFORMA O ESTADO DO AGENTE.

ACHADO 3 - GANHO EM LATÊNCIA: $8,6\times$ MENOR

JCT MÉDIO SAC-TOPSIS: 6,6 s $\times$ SAC-BASE: 57,1 s.

A COMBINAÇÃO TOPSIS + ESTADO COMPACTO + REWARD DEADLINE-AWARE

PRODUZ NÃO SÓ MAIS TAREFAS NO PRAZO, MAS LATÊNCIAS SUBSTANCIALMENTE MENORES.

HIERARQUIA: $\Delta_{\text{estado}} > \Delta_{\text{TOPSIS-decisão}} > \Delta_{\text{reward+feature}} \gg \Delta_{\text{ação-binária-estática}}$

AValiação Comparativa - Posições 1-12

658 CENÁRIOS V2X • ORDENADO POR SWQ DECRESCENTE • * = USA TOPSIS

$$SWQ(\pi) = \underbrace{\hat{\tau}(\pi)}_{\text{taxa de sucesso}} \times \underbrace{\frac{1}{|S|} \sum_{j \in S} \frac{D_j - JCT_j}{D_j}}_{\text{folga norm. média — só tarefas que cumpriram o prazo}}$$

⇒ política boa = cumpre prazo frequentemente e com margem

#	POLÍTICA	TAXA%	SWQ	JCT(s)	E(J)	CATEG.
1	sac-topsis*	45,29	0,1484	6,5	25,8	RL+TOPSIS
2	ppo-topsis*	42,75	0,1426	5,9	21,3	RL+TOPSIS
3	ppo-base	42,42	0,1418	6,0	24,4	RL-Base
4	ppo-topsis-compact*	42,42	0,1418	6,0	24,4	RL+TOPSIS
5	td3-base	42,42	0,1418	6,0	24,4	RL-Base
6	a2c-topsis*	42,42	0,1418	6,0	24,4	RL+TOPSIS
7	td3-topsis*	44,00	0,1388	3,7	23,0	RL+TOPSIS
8	sac-topsis-9d*	42,91	0,1386	5,9	21,0	RL+TOPSIS
9	sac-topsis-manhattan*	41,75	0,1385	11,8	23,7	RL+TOPSIS
10	sac-t1*	33,06	0,1270	11,5	26,1	RL+TOPSIS
11	greedy-unfair	41,38	0,1262	5,6	7,4	Heurística
12	sac-a1	39,21	0,1256	13,7	21,1	RL-Base

O SAC-TOPSIS LIDERA COM SWQ 0,1484. OS ALGORITMOS PPO, TD3 E A2C

CONVERGEM PARA 100% LOCAL (SWQ $\approx$ 0,1418). O GREEDY-UNFAIR COM

CONHECIMENTO PERFEITO DO ESTADO DA SIMULAÇÃO FICA ABAIXO DO SAC-TOPSIS

(0,1262).

AVALIAÇÃO COMPARATIVA - POSIÇÕES 13-25

658 CENÁRIOS V2X • ORDENADO POR SWQ DECRESCENTE • ★ = USA TOPSIS

#	POLÍTICA	TAXA%	SWQ	JCT(s)	E(J)	CATEG.
13	sac-r1	38,36	0,1223	13,4	31,4	RL-Base
14	sac-s1	28,81	0,1122	39,3	20,3	RL-Base
15	gini	35,34	0,1057	3,5	39,4	Heurística
16	dqn-topsis★	26,02	0,0997	21,7	24,0	RL+TOPSIS
17	jain	32,64	0,0973	3,6	39,3	Heurística
18	random	30,13	0,0950	17,9	27,3	Heurística
19	roundrobin	28,61	0,0905	17,9	31,1	Heurística
20	greedy	29,05	0,0823	29,3	6,2	Heurística
21	a2c-base	21,85	0,0772	13,7	48,9	RL-Base
22	dqn-base	22,15	0,0686	13,6	44,4	RL-Base
23	greedy-fair	20,65	0,0528	40,8	5,6	Heurística
24	sac-base	4,58	0,0119	57,1	24,2	RL-Base
25	rsu	1,93	0,0030	144,2	4,8	Heurística

O SAC-BASE (4,58%) COLAPSA POR USAR ESTADO 11D BRUTO SEM TOPSIS: 94,9% DAS AÇÕES SÃO V2V SEM CRITÉRIO. O DQN-TOPSIS (25,9%) MOSTRA QUE O TOPSIS TAMBÉM AJUDA ALGORITMOS BASEADOS EM VALOR, MAS EM MENOR ESCALA QUE COM O SAC.

RESULTADOS: OTIMIZAÇÃO DA FUNÇÃO OBJETIVO

O PROBLEMA FOI FORMULADO COMO POMDP COM FUNÇÃO OBJETIVO QUE ESTABELECE UM TRADE-OFF RÍGIDO: $w_s=0,60$ PARA TAXA DE SUCESSO DE DEADLINE E $w_c=0,40$ PARA CUSTO NORMALIZADO (MIN-MAX) DE LATÊNCIA E ENERGIA. AVALIADO EM 658 CENÁRIOS V2X, O SAC-TOPSIS ATINGIU $J(\pi)=0,2454$.

$$J(\pi) = \underbrace{w_s \cdot \mathbb{E}_\pi[1(\text{JCT}_i \leq D_i)]}_{\text{taxa de sucesso } (w_s=0,60)} - \underbrace{w_c \cdot \mathbb{E}_\pi \left[\frac{1}{2} \left(\frac{\text{JCT}_i}{\text{JCT}_i^{\max}} + \frac{E_i}{E_i^{\max}} \right) \right]}_{\text{custo norm. min-max } \in [0,1] \text{ } (w_c=0,40)}$$

$$\begin{aligned} \text{JCT}_i^{\max} &= \max(\widehat{\text{JCT}}_i^\ell, \widehat{\text{JCT}}_i^{\text{rsu}} \cdot \mathbf{1}_{\text{viável}}, \widehat{\text{JCT}}_i^{\text{v2v}} \cdot \mathbf{1}_{\text{viável}}) \\ E_i^{\max} &= \max(E_i^\ell, E_i^{\text{rsu}} \cdot \mathbf{1}_{\text{viável}}, E_i^{\text{v2v}} \cdot \mathbf{1}_{\text{viável}}) \quad \text{normalização} \\ &\quad \text{min-max por tarefa - garante custo } \in [0,1] \text{ para qualquer JCT} \end{aligned}$$

1º SAC-TOPSIS
 $J=0,2454$

TOP

2º PPO-TOPSIS
 $J=0,2344$

$J(\pi)$

3º PPO-Base
 $J=0,2295$

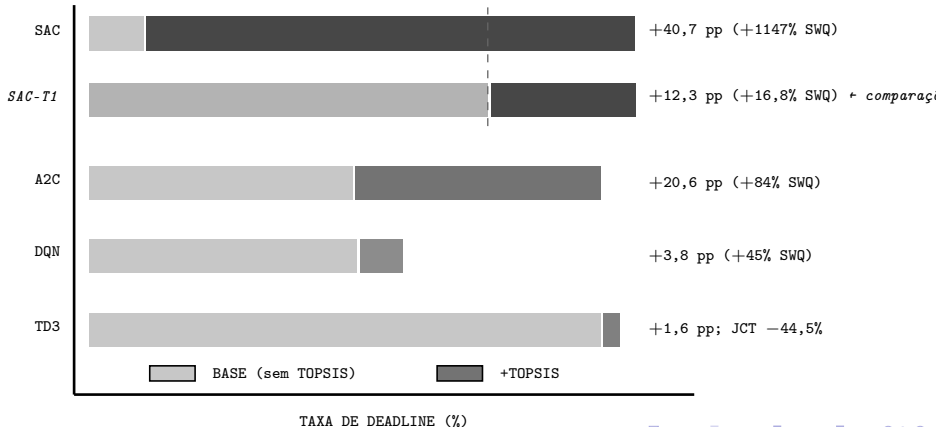
SAC-TOPSIS 0,2454
PPO-TOPSIS 0,2344
PPO-Base 0,2295

$J(\pi)$ (FUNÇÃO OBJETIVO)

O IMPACTO DO TOPSIS NOS ALGORITMOS DRL

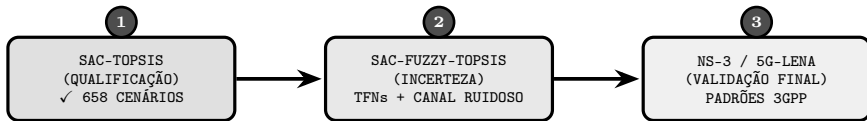
O TOPSIS ATUA COMO PRECONDICIONADOR AGNÓSTICO AO ALGORITMO. ACOPLADO AO A2C: +84% EM SWQ. AO DQN: +45%. AO TD3: JCT REDUZIDO EM 44,5%. AO SAC: +1147% (4,58% → 45,29%).

COMPARAÇÃO MAIS JUSTA: SAC-T1 (TOPSIS COMO FEATURE, 3 CLASSES) → SAC-TOPSIS (TOPSIS COMO DECISÃO, BINÁRIA): +12,3 PP, +16,8% SWQ. ISOLANDO O EFEITO DE DELEGAR O DESTINO AO TOPSIS.



PRÓXIMOS PASSOS: FUZZY-TOPSIS E NS-3

O AMBIENTE VEICULAR REAL É CAÓTICO: ESTIMATIVAS DE CANAL CHEGAM COM ATRASO E FILAS SOFREM FLUTUAÇÕES RÁPIDAS. O TOPSIS SERÁ SUBSTITUÍDO PELO FUZZY-TOPSIS: MODELANDO MEDIÇÕES COMO NÚMEROS FUZZY TRIANGULARES (TFNs), A ARQUITETURA ABSORVERÁ INCERTEZA LINGUÍSTICA, EVITANDO INVERSÕES INSTÁVEIS DE RANQUEAMENTO. O MODELO FINAL SAC-FUZZY-TOPSIS SERÁ VALIDADO NO SIMULADOR NS-3 (MÓDULO 5G-LENA), GARANTINDO ALTA FIDELIDADE FÍSICA AOS PADRÕES 3GPP.



ROBUSTEZ SOB INCERTEZA → FIDELIDADE FÍSICA AOS PADRÕES 3GPP